



SISTEMAS OPERATIVOS - TEORÍA
8 de Enero de 2026

Nombre _____ DNI _____
Apellidos _____ Grupo _____

Cuestión 1 (2 puntos). En un directorio de un sistema operativo tenemos 3 ficheros:

- Fichero1, tamaño lógico 3.000 bytes.
- Fichero2, tamaño lógico 54.000 bytes.
- Fichero3, tamaño lógico 4.500.000 bytes.

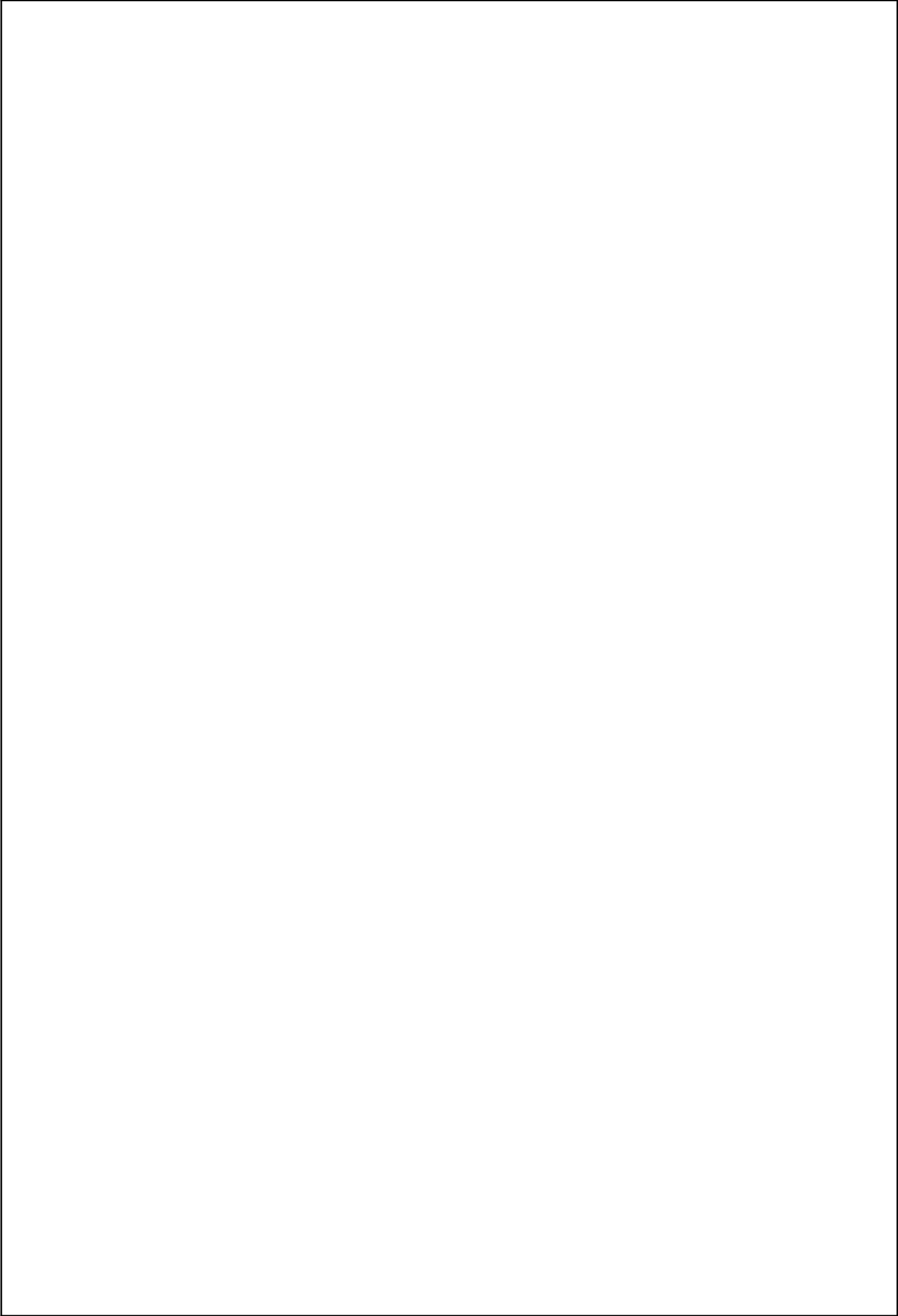
Asumiendo que el sistema de ficheros utiliza bloques de 4096 bytes, y que se utilizan 4 bytes para el identificador de bloque, responde razonadamente a los siguientes apartados:

- (0.5 puntos)** ¿Cuántos bloques tiene que reservar el sistema para cada uno de estos ficheros si el sistema de ficheros utiliza bloques indexados (tipo unix)?
- (0.5 puntos)** ¿Cuántos bloques se reservarán para cada fichero en un sistema de ficheros tipo FAT? ¿Cuántos bytes ocuparía la tabla FAT de dicho sistema?
- (0.5 puntos)** Indica qué estructuras del sistema de ficheros de bloques indexados se verían modificadas o actualizadas por la ejecución del comando *echo "Hola mundo" > Fichero4*. Describe y justifica en cada caso el cambio o actualización.
- (0.5 puntos)** Suponiendo que se ejecutan estos dos comandos:

Shell

```
ln -s Fichero4 Fichero5  
ln Fichero4 Fichero6
```

¿Cuántos ficheros se han creado en total con la ejecución de estos comandos? ¿Cuál será el contenido de los nuevos ficheros? Si después de esto se borra Fichero4, ¿cuál sería el resultado de ejecutar el comando *cat* sobre Fichero5 y Fichero6?



Cuestión 2 (2 puntos). Considere el siguiente código de un módulo del kernel de Linux que implementa un dispositivo de caracteres llamado `ticket_dispenser`. Este dispositivo gestiona un dispensador de turnos: cada vez que se lee del dispositivo, entrega un número consecutivo. Asuma que las cabeceras necesarias (`#include`) están presentes, que las funciones `init_module` y `cleanup_module` registran/liberan correctamente el dispositivo obteniendo el Major number 240, y que la estructura `file_operations` está correctamente asociada.

C/C++

```
static int current_ticket = 0;
static int device_open_count = 0;

static int dev_open(struct inode *inode, struct file *file) {
    if (device_open_count > 0) return -EBUSY; /* Solo permite un proceso a la vez */
    device_open_count++;
    return 0;
}

static int dev_release(struct inode *inode, struct file *file) {
    device_open_count--;
    return 0;
}

static ssize_t dev_read(struct file *filp, char *buffer, size_t len, loff_t *offset) {
    char msg[32];
    int msg_len;
    int bytes_to_copy;

    /* (A) Generar el mensaje */
    if (*offset == 0) current_ticket++;
    sprintf(msg, "Turno: %d\n", current_ticket);
    msg_len = strlen(msg);

    if (*offset >= msg_len) return 0;

    /* Calcular bytes a copiar */
    bytes_to_copy = msg_len - *offset;
    if (bytes_to_copy > len) bytes_to_copy = len;

    /* Copiar al usuario */
    if (copy_to_user(buffer, msg + *offset, bytes_to_copy)) return -EFAULT;

    *offset += bytes_to_copy;
    return bytes_to_copy;
}

static ssize_t dev_write(struct file *filp, const char *buff, size_t len, loff_t *off) {
    char input_cmd;
    if (len < 1) return -EFAULT;
    if (copy_from_user(&input_cmd, buff, 1))
        return -EFAULT;
    if (input_cmd == 'R' || input_cmd == 'r') {
        current_ticket = 0; /* Resetear contador */
        printk(KERN_INFO "Ticket dispenser reset.\n");
    }
    return len;
}
```

Responda a las siguientes preguntas, razonando su respuesta:

- a. **(0,5 puntos)** Indique el comando exacto de consola que debe ejecutar el administrador del sistema para crear el fichero especial de dispositivo `/dev/turnos` asociado a este driver, permitiendo lectura y escritura para todos los usuarios. Justifique los parámetros utilizados.
- b. **(0,75 puntos)** Suponga que el módulo acaba de cargarse y el fichero de dispositivo se ha creado correctamente. Un usuario ejecuta la siguiente secuencia de comandos en la shell. Indique qué salida exacta se muestra por pantalla y cuál es el valor de la variable `current_ticket` dentro del kernel al finalizar cada línea.
1. `cat /dev/turnos`
 2. `cat /dev/turnos`
 3. `echo "Reset" > /dev/turnos`
 4. `cat /dev/turnos`
- c. **(0,75 puntos)** Analice la función `dev_read`. Si quitamos la condición `if (*offset == 0)` en el punto (A) de forma que dejamos simplemente `current_ticket++;`, explique qué sucedería tras ejecutar el comando `cat /dev/turnos` varias veces consecutivas. ¿Qué comportamiento observaría el usuario y por qué?
- Nota:** Asuma que `cat` utiliza un buffer de 1KiB al hacer las lecturas sobre el fichero (en cada llamada a `read`).

Cuestión 3 (2 puntos). Considere el siguiente programa que se ejecuta en un sistema GNU/Linux, y suponga que todas las llamadas al sistema que invoca el programa tienen éxito:

```
C/C++
#include <fcntl.h>
#include <sys/stat.h>
#include <semaphore.h>
... resto de includes ...
#define N ...

int main(void){
    pid_t pid;
    pid_t ancestor = getpid();
    int i,fd;
    sem_t* s;
    char c;
    char buf[2];

    fd = open("log.txt", O_CREAT | O_RDWR | O_TRUNC, 0666);
    s = sem_open("MYSEM", O_CREAT, 0700, N);

    for (i = 0; i < 4; i++){
        pid = fork();
        if (pid == 0){
            sprintf(buf, "%i", i);
            write(fd, buf, strlen(buf));
            s = sem_open("MYSEM", 0);
        } else {
            write(1, "Hello\n", 6);
        }
    }

    sem_wait(s);
    while (wait(NULL) != -1) { }

    if (getpid() != ancestor){
        sem_close(s);
        exit(0);
    }

    sem_post(s);
    sem_close(s);
    sem_unlink("MYSEM");
    lseek(fd, 0, SEEK_SET);
    while(read(fd, &c, sizeof(char))>0)
        write(1, &c, sizeof(char));
    exit(0);
}
```

- (1 punto)** Indique cuál es el valor entero mínimo de N para que todos los procesos creados por el programa (y el propio padre) alcancen el primer bucle while del código. Justifique la respuesta ¿Cuántos procesos crea el programa (sin contar al proceso padre)? ¿Cuántos procesos pueden ejecutarse como máximo de forma concurrente?
- (0,75 puntos)** Indique un posible contenido final del fichero log.txt y responda a las siguientes preguntas:
 - ¿Pueden producirse sobrescrituras en el fichero al invocar concurrentemente write() desde varios procesos en el programa? Justifique la respuesta.
 - ¿Puede variar el contenido del fichero en distintas ejecuciones del programa? ¿Por qué? En caso afirmativo proponga algún cambio en el programa para garantizar que el contenido sea siempre el mismo, manteniendo el mismo tamaño que el fichero generado por el programa original, y sin alterar la llamada a write() sobre log.txt.

- c. **(0,25 puntos)** Indique lo que muestra el programa por pantalla. ¿Puede variar la salida en distintas ejecuciones? Justifique la respuesta

Cuestión 4. (2 puntos) Se te ha contratado para coordinar a los viajeros que intentan cruzar un río en un sentido concreto. Solo hay un barco, con capacidad para tres personas como máximo, y que cruza de una orilla a otra siempre llevando pasajeros (nunca vacío). El barco se hundirá si suben más de tres personas a la vez. Cada viajante se modela como un hilo POSIX independiente, que ejecuta la función siguiente, donde `id` es su identificador único y `dir` indica el sentido en el que quiere cruzar (0: de izquierda a derecha, 1: de derecha a izquierda):

```
C/C++
void traveler(int id, int dir)
{
    enter_boat(dir, id);
    cross_river(dir, id);
    exit_boat(dir, id);
}
```

Implemente las funciones `enter_boat` (**1 punto**) y `exit_boat` (**1 punto**) que permiten a los hilos sincronizarse para cruzar el río (función `cross_river`) cumpliendo las siguientes restricciones:

- Cuando el barco se encuentra en una orilla, sólo pueden subir los viajeros que estén en la misma orilla.
- Los viajeros subirán al barco en orden de llegada a cada orilla (es decir, el orden depende del sentido).
- Se permite la entrada al barco hasta llegar al aforo máximo (3 viajeros) o hasta que desembarca algún viajante, en cuyo caso se entiende que el barco ya ha llegado a la otra orilla.
- Cuando el barco llega a la otra orilla, los viajeros no pueden entrar al barco hasta que no hayan salido todos los del viaje anterior.

Para la resolución del problema se podrán emplear tanto semáforos, como mutex y variables de condición, indicando también las variables globales necesarias, así como su valor inicial. No es necesario incluir el código del programa `main()` que crea los hilos e inicializa los recursos de sincronización.

Cuestión 5 (2 puntos) Se ejecuta el código del programa `mem.c`, incluido a continuación, obteniendo como salida Child process (PID: 7569), Parent process PID: 7568. En una terminal diferente, se ejecutan una serie de comandos para obtener los mapas de memoria obteniendo los resultados reflejados a continuación del código.

```
C/C++
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/mman.h>
#include <pthread.h>

#define BUFSIZE 1024
char buf[BUFSIZE];
int x;

void *entry(void *arg) {
    printf("Child thread (PID: %d):\n", getpid());
    return NULL;
}

void signal_handler(int sig) {
    printf("Signal SIGUSR1 received (PID: %d)\n", getpid());
}

int main(int argc, char **argv) {
    pid_t pid;
    pthread_t thid;

    struct sigaction sa;
    sa.sa_handler = signal_handler;
    sa.sa_flags = 0;
    sigemptyset(&sa.sa_mask);
    sigaction(SIGUSR1, &sa, NULL);

    pid = fork();
    if (pid == 0) {
        printf("Child process (PID: %d), Parent process PID: %d\n", getpid(), getppid());
        pthread_create(&thid, NULL, entry, NULL);
        pause();
        pthread_join(thid, NULL);
        pause();
        printf("Process (PID: %d) has ended.\n", getpid());
        exit(EXIT_SUCCESS);
    } else {
        printf("I am the parent process (PID: %d), with a child PID: %d\n", getpid(), pid);
        wait(NULL);
        pause();
        printf("Process (PID: %d) has ended.\n", getpid());
    }
    return 0;
}
```

Salida de la terminal que se ejecuta en paralelo al programa:

```
$ cat /proc/7568/maps
5ebfbc635000-5ebfbc636000 r--p 00000000 00:43 63050394783252401 /workspaces/mem
5ebfbc636000-5ebfbc637000 r-xp 00001000 00:43 63050394783252401 /workspaces/mem
5ebfbc637000-5ebfbc638000 r--p 00002000 00:43 63050394783252401 /workspaces/mem
5ebfbc638000-5ebfbc639000 r--p 00002000 00:43 63050394783252401 /workspaces/mem
5ebfbc639000-5ebfbc63a000 rw-p 00003000 00:43 63050394783252401 /workspaces/mem
5ebfbdcff000-5ebfbdd20000 rw-p 00000000 00:00 0 [heap]
7d6c7780c000-7d6c7780f000 rw-p 00000000 00:00 0
7d6c7780f000-7d6c77831000 r--p 00000000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c77831000-7d6c7798a000 r-xp 00022000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c7798a000-7d6c779d9000 r--p 0017b000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c779d9000-7d6c779dd000 r--p 001c9000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c779dd000-7d6c779df000 rw-p 001cd000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c779df000-7d6c779e3000 rw-p 00000000 00:00 0
7d6c779e3000-7d6c779e9000 r--p 00000000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c779e9000-7d6c779f9000 r-xp 00006000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c779f9000-7d6c779ff000 r--p 00016000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c779ff000-7d6c77a00000 r--p 0001b000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c77a00000-7d6c77a01000 rw-p 0001c000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c77a01000-7d6c77a07000 rw-p 00000000 00:00 0
7d6c77a0d000-7d6c77a0e000 r--p 00000000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a0e000-7d6c77a2e000 r-xp 00001000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a2e000-7d6c77a36000 r--p 00021000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a37000-7d6c77a38000 r--p 00029000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a38000-7d6c77a39000 rw-p 0002a000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a39000-7d6c77a3a000 rw-p 00000000 00:00 0
7ffe0a998000-7ffe0a9b9000 rw-p 00000000 00:00 0 [stack]
7ffe0a9d7000-7ffe0a9db000 r--p 00000000 00:00 0 [vvar]
7ffe0a9db000-7ffe0a9dd000 r-xp 00000000 00:00 0 [vdso]

$ cat /proc/7569/maps
5ebfbc635000-5ebfbc636000 r--p 00000000 00:43 63050394783252401 /workspaces/mem
5ebfbc636000-5ebfbc637000 r-xp 00001000 00:43 63050394783252401 /workspaces/mem
5ebfbc637000-5ebfbc638000 r--p 00002000 00:43 63050394783252401 /workspaces/mem
5ebfbc638000-5ebfbc639000 r--p 00002000 00:43 63050394783252401 /workspaces/mem
5ebfbc639000-5ebfbc63a000 rw-p 00003000 00:43 63050394783252401 /workspaces/mem
5ebfbdcff000-5ebfbdd20000 rw-p 00000000 00:00 0 [heap]
7d6c7700b000-7d6c7700c000 ---p 00000000 00:00 0
7d6c7700c000-7d6c7780c000 rw-p 00000000 00:00 0
7d6c7780c000-7d6c7780f000 rw-p 00000000 00:00 0
7d6c7780f000-7d6c77831000 r--p 00000000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c77831000-7d6c7798a000 r-xp 00022000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c7798a000-7d6c779d9000 r--p 0017b000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c779d9000-7d6c779dd000 r--p 001c9000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c779dd000-7d6c779df000 rw-p 001cd000 08:40 45541 /lib/x86_64-linux-gnu/libc-2.31.so
7d6c779df000-7d6c779e3000 rw-p 00000000 00:00 0
7d6c779e3000-7d6c779e9000 r--p 00000000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c779e9000-7d6c779f9000 r-xp 00006000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c779f9000-7d6c779ff000 r--p 00016000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c779ff000-7d6c77a00000 r--p 0001b000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c77a00000-7d6c77a01000 rw-p 0001c000 08:40 45584 /lib/x86_64-linux-gnu/libpthread-2.31.so
7d6c77a01000-7d6c77a07000 rw-p 00000000 00:00 0
7d6c77a0d000-7d6c77a0e000 r--p 00000000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a0e000-7d6c77a2e000 r-xp 00001000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a2e000-7d6c77a36000 r--p 00021000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a37000-7d6c77a38000 r--p 00029000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a38000-7d6c77a39000 rw-p 0002a000 08:40 45534 /lib/x86_64-linux-gnu/ld-2.31.so
7d6c77a39000-7d6c77a3a000 rw-p 00000000 00:00 0
7ffe0a998000-7ffe0a9b9000 rw-p 00000000 00:00 0 [stack]
7ffe0a9d7000-7ffe0a9db000 r--p 00000000 00:00 0 [vvar]
7ffe0a9db000-7ffe0a9dd000 r-xp 00000000 00:00 0 [vdso]
```

Responda de manera razonada a las siguientes preguntas:

- a. **(1 punto)** Explique si el sistema está utilizando memoria contigua o virtual, identifique las regiones de memoria que puede reconocer, y analice las razones de coincidencia o discrepancia en los rangos de direcciones de cada región entre procesos. Comente el tipo de enlace utilizado con las bibliotecas: ¿es estático o dinámico?
- b. **(1 punto)** ¿Qué mapas de memoria (regiones) se obtendrán con la siguiente secuencia de comandos?

```
$ kill -SIGUSR1 7569
$ cat /proc/7568/maps
$ cat /proc/7569/maps
$ kill -SIGUSR1 7569
$ cat /proc/7569/maps
$ cat /proc/7568/maps
$ kill -SIGUSR1 7568
$ cat /proc/7568/maps
```